```
import logging
from datetime import date, datetime
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (Application, CommandHandler, CallbackQueryHandler,
                          MessageHandler, filters, ContextTypes, ConversationHan
from config import BOT_TOKEN, ADMIN_ID, CARD_NUMBER
from database import Database
from keyboards import (main_menu, admin_menu, dachas_kb, calendar_kb,
                       booking_confirm_kb, admin_booking_kb)

logging.basicConfig(level=logging.INFO)
db = Database()

(MAIN, SELECT_DACHA, START_DATE, END_DATE,
 ENTER_NAME, ENTER_PHONE, CONFIRM, RECEIPT,
 ADMIN, ADD_NAME, ADD_DESC, ADD_PRICE, ADD_LOCATION, ADD_PHOTO) = range(14)


async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data.clear()
    if update.effective_user.id == ADMIN_ID:
        await update.message.reply_text("👋 Admin paneliga xush kelibsiz!", reply
        return ADMIN
    await update.message.reply_text(
        "🏡 *Dacha Bron — Xush kelibsiz!*\n\nTabiat qo'ynida dam olish uchun dach
        parse_mode="Markdown", reply_markup=main_menu())
    return MAIN


async def main_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    if q.data == "dachas":
        dachas = db.get_dachas()
        if not dachas:
            await q.edit_message_text("😔 Hozircha dacha yo'q.", reply_markup=Inl
            return MAIN
        await q.edit_message_text("🏡 *Dachalarni tanlang:*", parse_mode="Markdow
        return SELECT_DACHA
    elif q.data == "my_bookings":
        bookings = db.get_user_bookings(update.effective_user.id)
        if not bookings:
            await q.edit_message_text("📋 Sizda bron yo'q.", reply_markup=InlineK
            return MAIN
```

```
        text = "📋 *Bronlaringiz:*\n\n"
        status_map = {"pending": "⏳", "confirmed": "✅", "rejected": "❌"}
        for b in bookings:
            text += f"{status_map.get(b['status'],'❓')} *{b['dacha_name']}*\n📅
        await q.edit_message_text(text, parse_mode="Markdown", reply_markup=Inlin
        return MAIN
    elif q.data == "back_main":
        await q.edit_message_text("🏠 *Dacha Bron*", parse_mode="Markdown", reply
        return MAIN


async def dacha_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    if q.data == "back_main":
        await q.edit_message_text("🏠 *Dacha Bron*", parse_mode="Markdown", reply
        return MAIN
    if q.data == "back_dachas":
        dachas = db.get_dachas()
        await q.edit_message_text("🏠 *Dachalarni tanlang:*", parse_mode="Markdow
        return SELECT_DACHA
    if q.data.startswith("d_"):
        dacha_id = int(q.data.split("_")[1])
        dacha = db.get_dacha(dacha_id)
        ctx.user_data['dacha_id'] = dacha_id
        text = (f"🏠 *{dacha['name']}*\n\n"
                f"📝 {dacha['description']}\n\n"
                f"📍 {dacha['location']}\n"
                f"💰 *{dacha['price']:,} so'm/kun*\n"
                f"💳 Avans (20%): *{int(dacha['price']*0.2):,} so'm*")
        kb = InlineKeyboardMarkup([
            [InlineKeyboardButton("📅 Bron qilish", callback_data=f"book_{dacha_i
            [InlineKeyboardButton("🔙 Orqaga", callback_data="back_dachas")]
        ])
        if dacha.get('photo_id'):
            try:
                await q.message.delete()
                await ctx.bot.send_photo(q.message.chat_id, dacha['photo_id'], ca
            except:
                await q.edit_message_text(text, parse_mode="Markdown", reply_mark
        else:
            await q.edit_message_text(text, parse_mode="Markdown", reply_markup=k
        return SELECT_DACHA
    if q.data.startswith("book_"):
        dacha_id = int(q.data.split("_")[1])
        ctx.user_data['dacha_id'] = dacha_id
        now = datetime.now()
```

```
        kb = calendar_kb(now.year, now.month, dacha_id, db, "start")
        try:
            await q.edit_message_caption("📅 *Kirish sanasini tanlang:*\n\n🟢 Bo'
        except:
            await q.edit_message_text("📅 *Kirish sanasini tanlang:*\n\n🟢 Bo'sh
        return START_DATE


async def start_date_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    if q.data == "ignore":
        return START_DATE
    if q.data == "back_dachas":
        dachas = db.get_dachas()
        try:
            await q.edit_message_caption("🏡 *Dachalarni tanlang:*", parse_mode="
        except:
            await q.edit_message_text("🏡 *Dachalarni tanlang:*", parse_mode="Mar
        return SELECT_DACHA
    if q.data.startswith("cp_") or q.data.startswith("cn_"):
        parts = q.data.split("_")
        direction, year, month, stage, dacha_id = parts[0], int(parts[1]), int(pa
        if direction == "cp":
            month -= 1
            if month == 0: month, year = 12, year-1
        else:
            month += 1
            if month == 13: month, year = 1, year+1
        kb = calendar_kb(year, month, dacha_id, db, stage)
        try:
            await q.edit_message_caption("📅 *Kirish sanasini tanlang:*\n\n🟢 Bo'
        except:
            await q.edit_message_text("📅 *Kirish sanasini tanlang:*\n\n🟢 Bo'sh
        return START_DATE
    if q.data.startswith("ds_"):
        parts = q.data.split("_")
        year, month, day, dacha_id = int(parts[1]), int(parts[2]), int(parts[3]),
        selected = date(year, month, day)
        if selected < date.today():
            await q.answer("⚠️ O'tgan sana!", show_alert=True)
            return START_DATE
        if db.is_booked(dacha_id, selected):
            await q.answer("🔴 Bu sana band!", show_alert=True)
            return START_DATE
        ctx.user_data['start_date'] = selected
        ctx.user_data['dacha_id'] = dacha_id
```

```
        kb = calendar_kb(year, month, dacha_id, db, "end", start_date=selected)
        try:
            await q.edit_message_caption(
                f"✅ Kirish: *{selected.strftime('%d.%m.%Y')}*\n\n📅 *Chiqish sar
                parse_mode="Markdown", reply_markup=kb)
        except:
            await q.edit_message_text(
                f"✅ Kirish: *{selected.strftime('%d.%m.%Y')}*\n\n📅 *Chiqish sar
                parse_mode="Markdown", reply_markup=kb)
        return END_DATE


async def end_date_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    if q.data == "ignore":
        return END_DATE
    dacha_id = ctx.user_data.get('dacha_id')
    start_date = ctx.user_data.get('start_date')
    if q.data.startswith("cp_") or q.data.startswith("cn_"):
        parts = q.data.split("_")
        direction, year, month = parts[0], int(parts[1]), int(parts[2])
        if direction == "cp":
            month -= 1
            if month == 0: month, year = 12, year-1
        else:
            month += 1
            if month == 13: month, year = 1, year+1
        kb = calendar_kb(year, month, dacha_id, db, "end", start_date=start_date)
        try:
            await q.edit_message_caption(f"✅ Kirish: *{start_date.strftime('%d.%
        except:
            await q.edit_message_text(f"✅ Kirish: *{start_date.strftime('%d.%m.%
        return END_DATE
    if q.data.startswith("back_start_"):
        dacha_id = int(q.data.split("_")[2])
        now = datetime.now()
        kb = calendar_kb(now.year, now.month, dacha_id, db, "start")
        try:
            await q.edit_message_caption("📅 *Kirish sanasini tanlang:*\n\n🟢 Bo'
        except:
            await q.edit_message_text("📅 *Kirish sanasini tanlang:*\n\n🟢 Bo'sh
        return START_DATE
    if q.data.startswith("de_"):
        parts = q.data.split("_")
        year, month, day = int(parts[1]), int(parts[2]), int(parts[3])
        end_date = date(year, month, day)
```

```
        days = (end_date - start_date).days
        if days <= 0:
            await q.answer("⚠️ Chiqish kirish sanasidan keyin bo'lishi kerak!", s
            return END_DATE
        if days > 4:
            await q.answer("⚠️ Maksimal 4 kun!", show_alert=True)
            return END_DATE
        for i in range(days):
            check = date.fromordinal(start_date.toordinal() + i)
            if db.is_booked(dacha_id, check):
                await q.answer(f"🔴 {check.strftime('%d.%m')} band!", show_alert=
                return END_DATE
        ctx.user_data['end_date'] = end_date
        ctx.user_data['days'] = days
        try:
            await q.edit_message_caption("👤 *Ism va familiyangizni kiriting:*\n\
        except:
            await q.edit_message_text("👤 *Ism va familiyangizni kiriting:*\n\n_M
        return ENTER_NAME


async def name_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    name = update.message.text.strip()
    if len(name) < 3:
        await update.message.reply_text("⚠️ To'liq ism familiya kiriting.")
        return ENTER_NAME
    ctx.user_data['name'] = name
    await update.message.reply_text("📞 *Telefon raqamingizni kiriting:*\n\n_Misc
    return ENTER_PHONE


async def phone_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    phone = update.message.text.strip().replace(" ", "").replace("-", "")
    if not any(phone.startswith(p) for p in ["+998", "998", "0"]):
        await update.message.reply_text("⚠️ To'g'ri telefon raqam kiriting.\n_Mis
        return ENTER_PHONE
    ctx.user_data['phone'] = phone
    dacha = db.get_dacha(ctx.user_data['dacha_id'])
    days = ctx.user_data['days']
    total = dacha['price'] * days
    advance = int(total * 0.2)
    ctx.user_data['total'] = total
    ctx.user_data['advance'] = advance
    text = (f"📋 *Bron ma'lumotlari:*\n\n"
            f"🏠 {dacha['name']}\n"
            f"📅 {ctx.user_data['start_date'].strftime('%d.%m.%Y')} → {ctx.user_d
            f"🌙 {days} kun\n"
```

```
            f"👤 {ctx.user_data['name']}\n"
            f"📞 {phone}\n\n"
            f"💰 Jami: *{total:,} so'm*\n"
            f"💳 Avans (20%): *{advance:,} so'm*")
    await update.message.reply_text(text, parse_mode="Markdown", reply_markup=boo
    return CONFIRM


async def confirm_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    if q.data == "cancel":
        ctx.user_data.clear()
        await q.edit_message_text("❌ Bekor qilindi.", reply_markup=main_menu())
        return MAIN
    if q.data == "do_confirm":
        await q.edit_message_text(
            f"💳 *To'lov ma'lumotlari:*\n\n"
            f"Avans miqdori: *{ctx.user_data['advance']:,} so'm*\n\n"
            f"💳 Karta raqam:\n`{CARD_NUMBER}`\n\n"
            f"_(Karta raqamni bosib nusxa oling)_\n\n"
            f"To'lovdan so'ng chek rasmini yuboring 👇",
            parse_mode="Markdown",
            reply_markup=InlineKeyboardMarkup([
                [InlineKeyboardButton("📸 Chek rasmini yuborish", callback_data="
                [InlineKeyboardButton("❌ Bekor qilish", callback_data="cancel")]
            ])
        )
        return RECEIPT


async def receipt_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if update.callback_query:
        q = update.callback_query
        await q.answer()
        if q.data == "cancel":
            ctx.user_data.clear()
            await q.edit_message_text("❌ Bekor qilindi.", reply_markup=main_menu
            return MAIN
        if q.data == "send_receipt":
            await q.edit_message_text("📸 *Chek rasmini yuboring:*", parse_mode="
        return RECEIPT
    if not update.message.photo:
        await update.message.reply_text("⚠️ Iltimos, chek *rasmini* yuboring.", p
        return RECEIPT
    photo = update.message.photo[-1]
    user_id = update.effective_user.id
```

```
    username = update.effective_user.username or "Yo'q"
    dacha = db.get_dacha(ctx.user_data['dacha_id'])
    booking_id = db.create_booking(
        user_id=user_id, dacha_id=ctx.user_data['dacha_id'],
        start_date=ctx.user_data['start_date'], end_date=ctx.user_data['end_date'
        name=ctx.user_data['name'], phone=ctx.user_data['phone'],
        total_price=ctx.user_data['total'], advance=ctx.user_data['advance'],
        receipt_photo_id=photo.file_id
    )
    admin_text = (f"🔔 *Yangi bron #{booking_id}*\n\n"
                  f"🏡 {dacha['name']}\n"
                  f"📅 {ctx.user_data['start_date']} → {ctx.user_data['end_date']
                  f"👤 {ctx.user_data['name']}\n"
                  f"📞 {ctx.user_data['phone']}\n"
                  f"💬 @{username}\n\n"
                  f"💰 Jami: *{ctx.user_data['total']:,} so'm*\n"
                  f"💳 Avans: *{ctx.user_data['advance']:,} so'm*")
    try:
        await ctx.bot.send_photo(ADMIN_ID, photo.file_id, caption=admin_text,
                                 parse_mode="Markdown", reply_markup=admin_bookin
    except:
        pass
    await update.message.reply_text(
        f"✅ *Chek yuborildi!*\n\nBron #{booking_id} ko'rib chiqilmoqda.\nAdmin t
        parse_mode="Markdown", reply_markup=main_menu())
    ctx.user_data.clear()
    return MAIN


async def admin_handler(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    if q.data == "add_dacha":
        await q.edit_message_text("1️⃣ Dacha nomini kiriting:")
        return ADD_NAME
    elif q.data == "manage_dachas":
        dachas = db.get_dachas()
        if not dachas:
            await q.edit_message_text("Dacha yo'q.", reply_markup=admin_menu())
            return ADMIN
        kb = [[InlineKeyboardButton(f"🗑 {d['name']}", callback_data=f"del_{d['id
        kb.append([InlineKeyboardButton("🔙 Orqaga", callback_data="admin_back")]
        text = "🏡 *Dachalar:*\n\n" + "\n".join(f"• {d['name']} — {d['price']:,}
        await q.edit_message_text(text, parse_mode="Markdown", reply_markup=Inlin
        return ADMIN
    elif q.data == "all_bookings":
        bookings = db.get_all_bookings()
```

```
    if not bookings:
        await q.edit_message_text("Bron yo'q.", reply_markup=admin_menu())
        return ADMIN
    status_map = {"pending": "⏳", "confirmed": "✅", "rejected": "❌"}
    text = "📋 *Bronlar:*\n\n"
    kb = []
    for b in bookings:
        text += f"{status_map.get(b['status'],'❓')} #{b['id']} {b['dacha_nam
        if b['status'] == 'pending':
            kb.append([InlineKeyboardButton(f"✅ #{b['id']}", callback_data=f
                       InlineKeyboardButton(f"❌ #{b['id']}", callback_data=f
    kb.append([InlineKeyboardButton("🔙 Orqaga", callback_data="admin_back")]
    await q.edit_message_text(text, parse_mode="Markdown", reply_markup=Inlin
    return ADMIN
elif q.data == "stats":
    s = db.stats()
    text = (f"📊 *Statistika:*\n\n"
            f"🏠 Dachalar: *{s['dachas']}*\n"
            f"📋 Jami bronlar: *{s['total']}*\n"
            f"✅ Tasdiqlangan: *{s['confirmed']}*\n"
            f"⏳ Kutilmoqda: *{s['pending']}*\n"
            f"❌ Rad etilgan: *{s['rejected']}*\n"
            f"💰 Daromad: *{s['revenue']:,} so'm*")
    await q.edit_message_text(text, parse_mode="Markdown",
                              reply_markup=InlineKeyboardMarkup([[InlineKeybo
    return ADMIN
elif q.data.startswith("approve_"):
    booking_id = int(q.data.split("_")[1])
    b = db.get_booking(booking_id)
    if b:
        db.update_status(booking_id, "confirmed")
        try:
            await ctx.bot.send_message(b['user_id'],
                f"🎉 Bron #{booking_id} tasdiqlandi!\n\n🏠 {b['dacha_name']}\
        except: pass
        await q.answer("✅ Tasdiqlandi!", show_alert=True)
        try:
            await q.edit_message_caption(q.message.caption + "\n\n✅ *TASDIQI
        except: pass
    return ADMIN
elif q.data.startswith("reject_"):
    booking_id = int(q.data.split("_")[1])
    b = db.get_booking(booking_id)
    if b:
        db.update_status(booking_id, "rejected")
        try:
            await ctx.bot.send_message(b['user_id'], f"😔 Bron #{booking_id}
```

```
            except: pass
            await q.answer("❌ Rad etildi!", show_alert=True)
            try:
                await q.edit_message_caption(q.message.caption + "\n\n❌ *RAD ETI
            except: pass
        return ADMIN
    elif q.data.startswith("del_"):
        dacha_id = int(q.data.split("_")[1])
        d = db.get_dacha(dacha_id)
        await q.edit_message_text(f"⚠️ *{d['name']}* ni o'chirmoqchimisiz?", pars
                                  reply_markup=InlineKeyboardMarkup([
                                      [InlineKeyboardButton("✅ Ha", callback_dat
                                       InlineKeyboardButton("❌ Yo'q", callback_d
                                  ]))
        return ADMIN
    elif q.data.startswith("confirm_del_"):
        dacha_id = int(q.data.split("_")[2])
        db.delete_dacha(dacha_id)
        await q.answer("🗑️ O'chirildi!", show_alert=True)
        await q.edit_message_text("👋 Admin paneliga xush kelibsiz!", reply_marku
        return ADMIN
    elif q.data == "admin_back":
        await q.edit_message_text("👋 Admin paneliga xush kelibsiz!", reply_marku
        return ADMIN


async def add_name(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if update.callback_query:
        await update.callback_query.answer()
        return ADD_NAME
    ctx.user_data['new_name'] = update.message.text.strip()
    await update.message.reply_text("2️⃣ Tavsifini kiriting:\n_Misol: 3 xonali, hc
    return ADD_DESC


async def add_desc(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['new_desc'] = update.message.text.strip()
    await update.message.reply_text("3️⃣ Kunlik narxini kiriting (so'mda):\n_Misol
    return ADD_PRICE


async def add_price(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    try:
        price = int(update.message.text.strip().replace(",", "").replace(" ", "")
        if price <= 0: raise ValueError
    except:
        await update.message.reply_text("⚠️ To'g'ri narx kiriting (faqat raqam)."
```

```
        return ADD_PRICE
    ctx.user_data['new_price'] = price
    await update.message.reply_text("4️⃣ Manzilini kiriting:\n_Misol: Toshkent vil
    return ADD_LOCATION


async def add_location(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['new_location'] = update.message.text.strip()
    await update.message.reply_text("5️⃣ Rasmini yuboring:",
        reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("⏭️ Rasmisiz dav
    return ADD_PHOTO


async def add_photo(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    photo_id = None
    if update.callback_query:
        await update.callback_query.answer()
        if update.callback_query.data != "skip_photo":
            return ADD_PHOTO
    elif update.message and update.message.photo:
        photo_id = update.message.photo[-1].file_id
    else:
        await update.message.reply_text("⚠️ Rasm yuboring yoki 'Rasmisiz davom et
        return ADD_PHOTO
    db.add_dacha(ctx.user_data['new_name'], ctx.user_data['new_desc'],
                 ctx.user_data['new_price'], ctx.user_data['new_location'], photo
    text = (f"✅ *Dacha qo'shildi!*\n\n"
            f"🏡 {ctx.user_data['new_name']}\n"
            f"📍 {ctx.user_data['new_location']}\n"
            f"💰 {ctx.user_data['new_price']:,} so'm/kun")
    ctx.user_data.clear()
    if update.callback_query:
        await update.callback_query.edit_message_text(text, parse_mode="Markdown"
    else:
        await update.message.reply_text(text, parse_mode="Markdown", reply_markup
    return ADMIN


def main():
    app = Application.builder().token(BOT_TOKEN).build()
    conv = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            MAIN: [CallbackQueryHandler(main_handler)],
            SELECT_DACHA: [CallbackQueryHandler(dacha_handler)],
            START_DATE: [CallbackQueryHandler(start_date_handler)],
            END_DATE: [CallbackQueryHandler(end_date_handler)],
```

```
            ENTER_NAME: [MessageHandler(filters.TEXT & ~filters.COMMAND, name_han
            ENTER_PHONE: [MessageHandler(filters.TEXT & ~filters.COMMAND, phone_h
            CONFIRM: [CallbackQueryHandler(confirm_handler)],
            RECEIPT: [
                MessageHandler(filters.PHOTO, receipt_handler),
                MessageHandler(filters.TEXT & ~filters.COMMAND, receipt_handler),
                CallbackQueryHandler(receipt_handler),
            ],
            ADMIN: [CallbackQueryHandler(admin_handler)],
            ADD_NAME: [MessageHandler(filters.TEXT & ~filters.COMMAND, add_name),
            ADD_DESC: [MessageHandler(filters.TEXT & ~filters.COMMAND, add_desc)]
            ADD_PRICE: [MessageHandler(filters.TEXT & ~filters.COMMAND, add_price
            ADD_LOCATION: [MessageHandler(filters.TEXT & ~filters.COMMAND, add_lo
            ADD_PHOTO: [MessageHandler(filters.PHOTO, add_photo), CallbackQueryHa
        },
        fallbacks=[CommandHandler("start", start)],
        allow_reentry=True,
    )
    app.add_handler(conv)
    print("🤖 Bot ishga tushdi...")
    app.run_polling()


if __name__ == "__main__":
    main()
```